Computer Vision:

Homework-4

Bilal Ahmed
ahmedb@purdue.edu

Theory Qestion

What is the theoretical reason for why the LoG of an image can be computed as a DoG?

Answer: LoG refers to Laplacian of Gaussian of an image. As the name suggests it is computed by first convolving the image *f(x,y)* with the Guassian function *g(x,y)* and then taking applying the Laplacian operator ∇^2. Where Guassian function is given as;

$$g(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

and Laplacian operator as;

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

The result of convolving an image *f(x,y)* with Gaussian function *g(x,y)* with width of center lobe specified by σ is refered to as σ-smoothed image and it represented as;

$$\text{ff}(x,y,\sigma) = f(x,y) * g(x,y)$$

So we can write LoG as;

$$\begin{aligned} LoG(f(x,y)) &= \nabla^2 \text{ff}(x,y,\sigma) \\ &= \nabla^2 (f(x,y) * g(x,y)) \\ &= f(x,y) * \nabla^2 g(x,y) \\ &= f(x,y) * h(x,y) \end{aligned}$$

where,

$$\begin{aligned} h(x,y) &= \nabla^2 g(x,y) \\ &= \nabla^2 g(x,y) \\ &= \frac{-1}{2\pi\sigma^4}(2 - \frac{x^2+y^2}{\sigma^2}) e^{-\frac{x^2+y^2}{\sigma^2}} \end{aligned}$$

DoG is an approximation to the derivative of σ-smoothed image w.r.t. σ. Let's write the exact expression for this derivative;

$$\frac{\partial}{\partial\sigma}\text{ff}(x,y,\sigma) = \frac{\partial}{\partial\sigma} f(x,y) * g(x,y)$$

$$= f(x,y) * \frac{\partial}{\partial\sigma} g(x,y)$$

$$= f(x,y) * \frac{-\sigma}{2\pi\sigma^4}(2 - \frac{x^2+y^2}{\sigma^2}) e^{-\frac{x^2+y^2}{\sigma^2}}$$

$$= \sigma f(x,y) * h(x,y)$$

That means we can write;

$$\frac{\partial}{\partial\sigma}\text{ff}(x,y,\sigma) = \sigma\nabla^2\text{ff}(x,y,\sigma)$$

The result shown above is fundamental result of scale space analysis and it proves that LoG of an image can be computed as a DoG.

why computing the LoG of an image as a DoG is computationally much more efficient for the same value of σ?

Answer: DoG involves convolution of the image with Gaussian kernel at two nearby values of σ and taking the difference. Kernel for getting the σ-smoothed image is separable in x and y, so instead of doing 2-D convolution, we can get away by doing two 1-D convolutions. Contrary to this kernel for LoG is not separable in x and y. In addition to this radius of center lobe in of 1D kernel is smaller than the radius of center lobe in 2D. So another advantage of doing 1D convolutions instead of 2D convolution is that we need kernels having smaller number of pixels.
For example for $\sigma = \sqrt{2}$, radius of center lobe in 2D is $\sqrt{2}\sigma = 2$ and if we choose to extend the kernel width upto $3 \times radius$ on each side, total size of kernel would be 13×13. Whereas for 1D for the same value of $\sigma = \sqrt{2}$, radius of center lobe is σ, so extending kernel width to 3 times the radius on each side would give us 9 samples for 1D convolution. Therefore, computing LoG as DoG is computationally more efficient.

Programming Tasks

Task-1: Harris Corner Detection

Corners are the parts of the image where the pixel intensities changes in every direction, these might be of interest to us for the applications likes stereo and tracking objects. Harris detection provides a mathematical way of identifying the corner using the gradients in x and y directions. Harris corner detection uses gray scale images, so first of all input image is converted to gray scale image. Here is the step-by-step algorithm;

- Compute the gradients of the image along x direction using using haar window, to get an image, say d_x. For M=4, here is what $M \times M$ haar window looks like;

$$W_x = \begin{bmatrix} -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \end{bmatrix}$$

- Similarly compute the gradients of the image along y direction using using haar window, to get an image, say d_y. For M=4, here is what $M \times M$ haar window for y-axis, W_y looks like;

$$W_y = \begin{bmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & -1 & -1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

- Then following matrix is computed using the result of above two operations, at each pixel location, using the $5\sigma \times 5\sigma$ window.

$$C = \begin{bmatrix} \sum d_x^2 & \sum d_x d_y \\ \sum d_x d_y & \sum d_x^2 \end{bmatrix}$$

- Using C, following response function is computed;

$$R = \det(C) - k\,(\mathrm{Tr}(C))^2$$

k was set equal to 0.05.

- In order to improve the noise rejection, a pixel is added to the list of interest points only if (1) its response is greater than the mean response and (2) its response is maximum in the window of $10\sigma \times 10\sigma$, refered to as non-maximum suppression. In addition, I limit the number of interest points in an image to a maximum of 500.

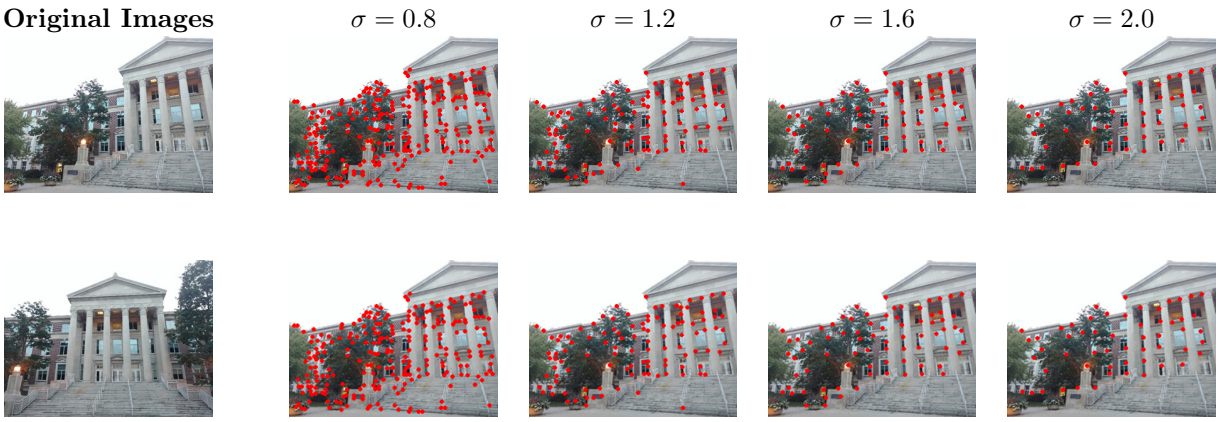


Figure 1: Original images (leftmost) and transformed images for different values of σ from the images of 'hovde'.

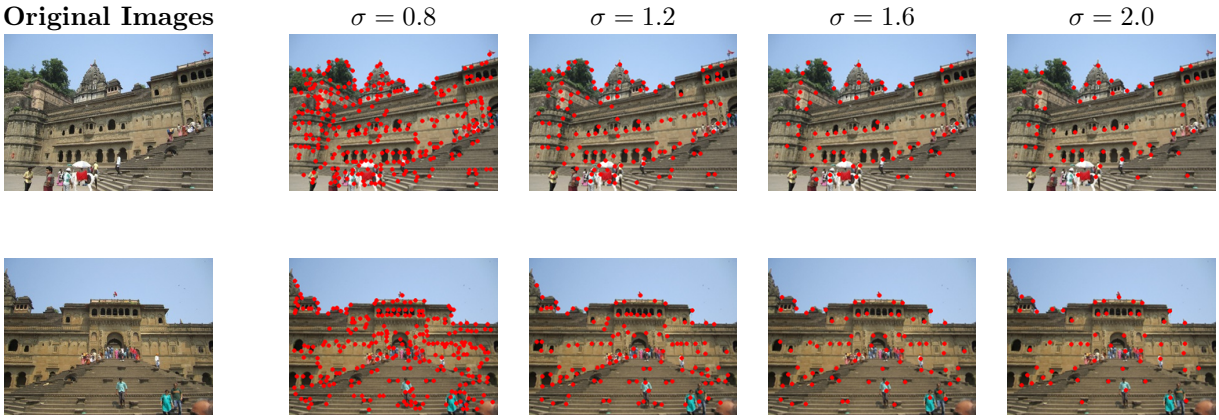


Figure 2: Original images (leftmost) and transformed images for different values of σ from the images of 'temple'.

Establishing correspondence between pair of images

Harris corner detection gives interest points that are invariant to in-plan rotations. So given two views, we can use these interest points to matches corresponding points. But to be able to match interest points, we need descriptors for these interest points. Advanced algorithms like SIFT and SURF give descriptors in addition to location of interest points. But for Harris corner detection, we can use the pixels in the window around interest points and use them as descriptors. Next, we need a way to match these descriptors.

Figure 3: SSD correspondences for different values of σ from the set 'hovde'.

Figure 4: NCC correspondences for different values of σ from the set 'hovde'.

Sum of Squared Differences (SSD)

We can compute SSD as measure of distance between descriptors, using the following expression;

$$\text{SSD} = \sum_i \sum_j |f_1(i,j) - f_2(i,j)|^2$$

This measure is un-normalized and smaller values mean higher similarity.

Normalized Cross Correlation

Another measure that can be used to quantify similarity mathematically is NCC.

$$\text{NCC} = \frac{\sum_i \sum_j (f_1(i,j) - \mu_1)(f_2(i,j) - \mu_2)}{\sqrt{\sum_i \sum_j (f_1(i,j) - \mu_1)^2} \sqrt{\sum_i \sum_j (f_2(i,j) - \mu_2)^2}}$$

Here μ_1 and μ_2 are means of descriptors 1 and 2 respectively. This measure is better in a sense that it is normalized and value of 1 would mean perfect matching and 0 would mean no matching at all.

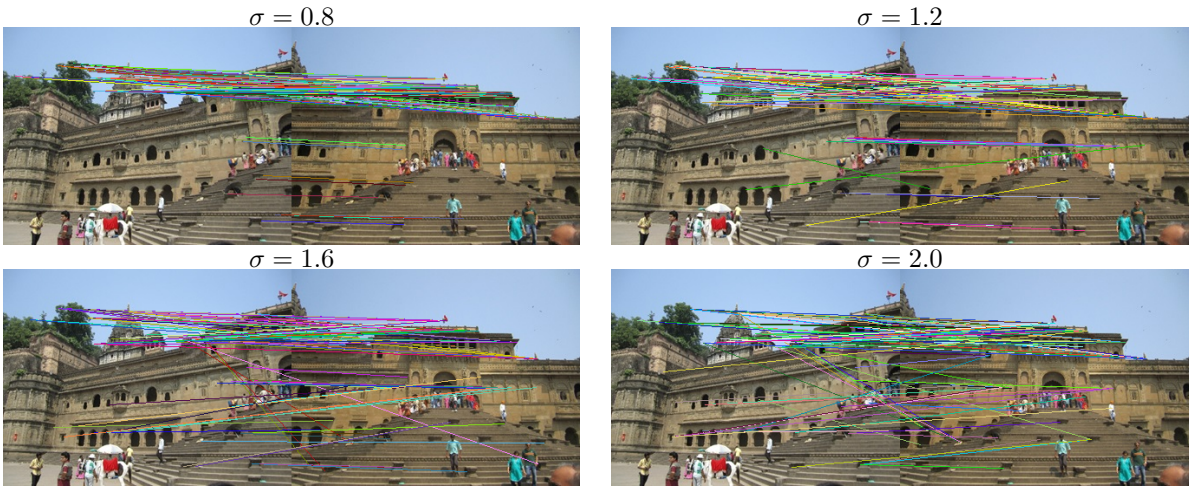


Figure 5: SSD correspondences for different values of σ from the set 'temple'.

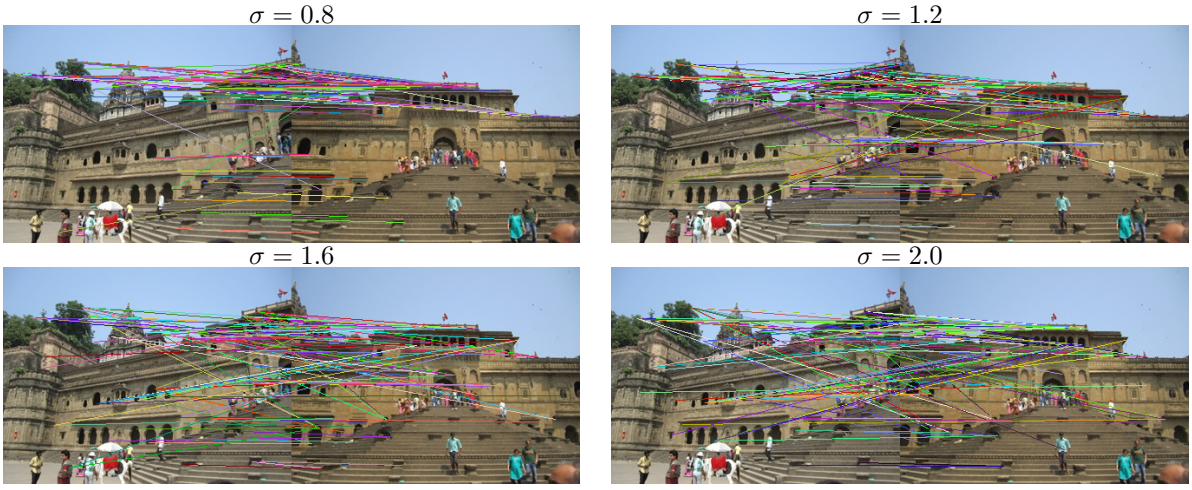


Figure 6: NCC correspondences for different values of σ from the set 'temple'.

For both of these measures, I threshold the values to retain the top 100 strongest matches.

Now I present the results of application of Harris corner detection, and interest points matching using both SSD and NCC on pairs of images.

Observation on results of Harris Corner Detection

The scale of interest points detected by Harris corner detection depends on the value of σ used. For small values, a large number of interest points are detected by the algorithm. So it depends on how fine are the details that we want to capture. For example for both temple and hovde images, $\sigma = 2$ gives good interest points.

Observation on SSD/NCC

Both SSD and NCC give comparable performance. NCC is better in that it is normalized whereas for SSD we need to get relatively good feature by comparing against all given scores.

Scale-invariant feature transform (SIFT)

Harris corner detection is invariant to rotations but it has a few drawbacks. Firstly, it is not invariant to scale of interest points in the image and secondly it does not provide descriptive vectors. We did use window of pixels around interest points but that is not a very useful descriptive vector, especially for features at different scales. Scale-invariant feature transform uses scale-space representation to extract and localize interest points and also gives 128 dimenional descriptive vectors. I have used openCV implementation of SIFT here to capture interest points and establish correspondences between image pairs. Figure 7 shows results for 'hovde' and 'temple' image pairs.

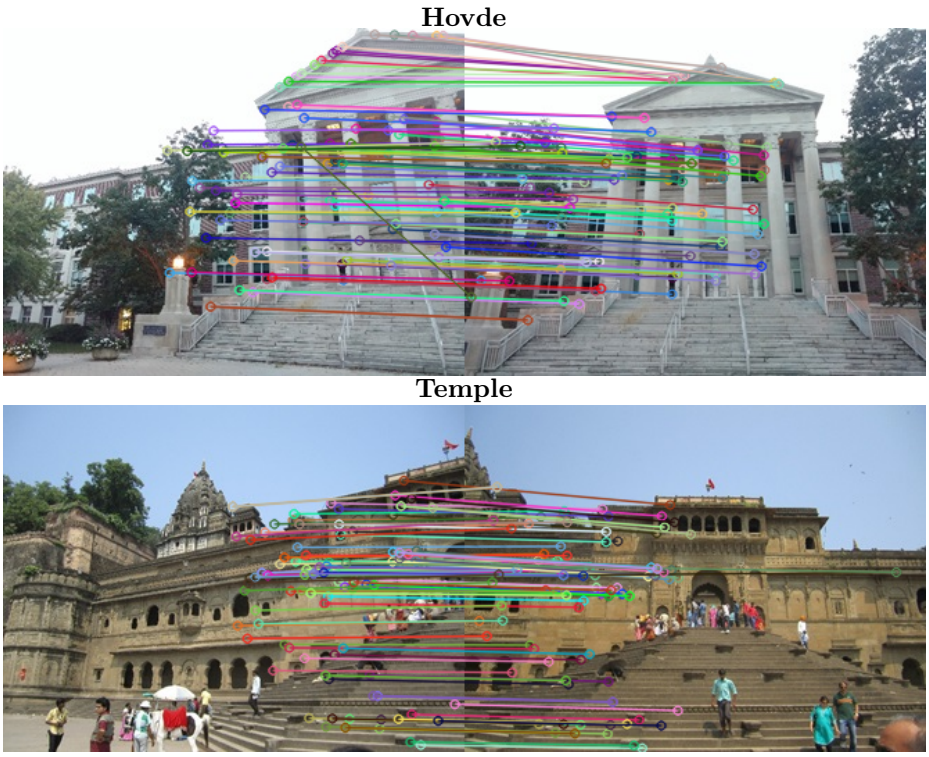


Figure 7: Interest points and correspondences using SIFT for the 'Hovde' and 'Temple' pairs.

Using SuperPoint and SuperGlue

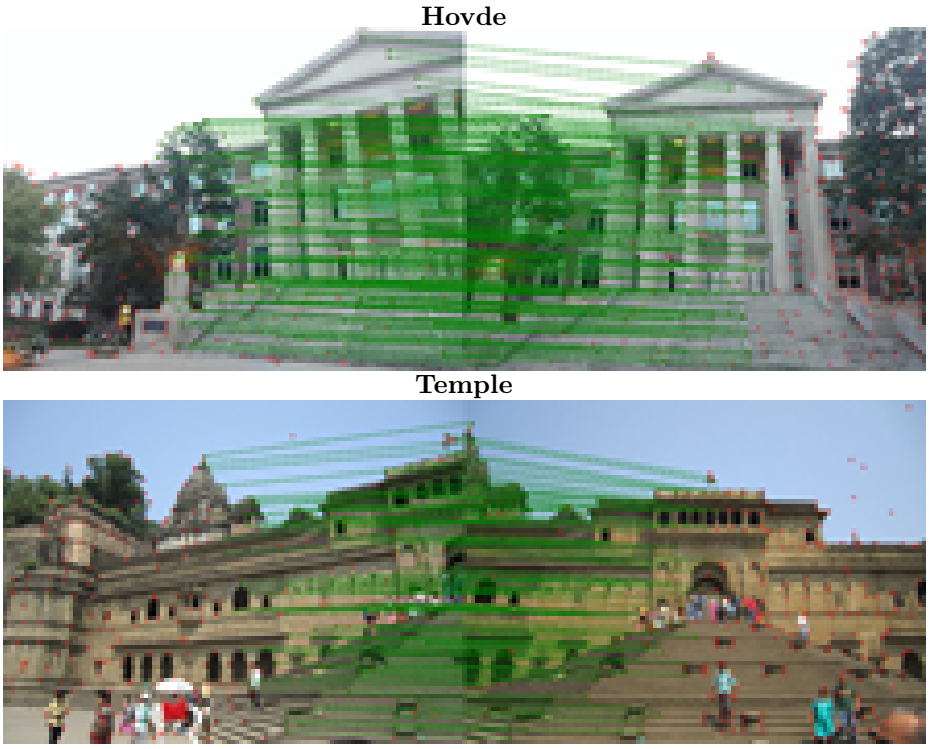


Figure 8: Result of using SuperPoint and SuperGLue for the 'Hovde' and 'Temple' pairs.

Observation on Harris/SIFT/SuperPoint

Harris corner detection gives interest points but it does not give us descriptive features, so getting good correspondences is difficult, where SIFT give descriptive features and thus we get good correspondences. Also results of deep learning based SuperPoint and SuperGlue are very good.

Task-2

All the above results for two more pair of images, image pairs of 'fuel' building and 'painting' in ECE.

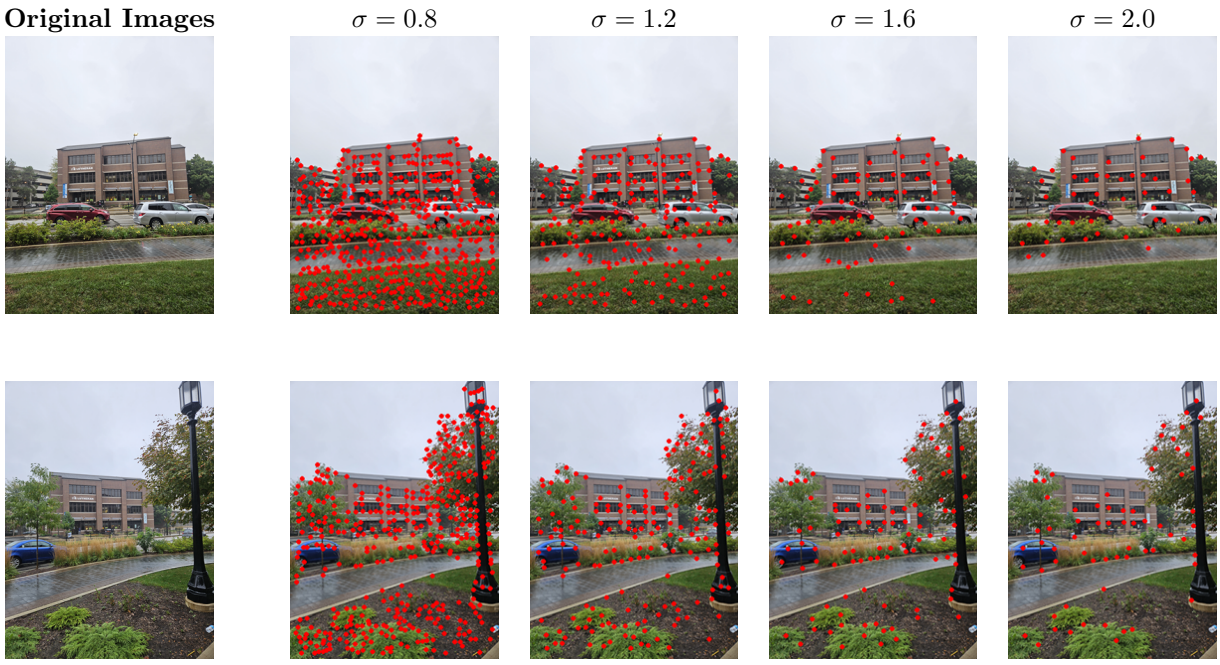


Figure 9: Original images (leftmost) and transformed images for different values of σ from the images of 'fuel'.

Python Code

```
import cv2
import pathlib
import computer_vision

import matplotlib.pylab as plt
import matplotlib as mpl
import numpy as np
import math


def read_image(subdir, image_name):
    """Reads images from the subdir"""
    path = pathlib.Path(computer_vision.__path__[0])
    image_path = path.parents[0] / 'images' /subdir/image_name
    img_bgr = cv2.imread(image_path)
    # Convert the image from BGR to RGB format
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    return img_rgb

def save_image(image, subdir, image_name):
    """Saves image to the subdir"""
    path = pathlib.Path(computer_vision.__path__[0])
    image_path = path.parents[0] / 'images' /subdir/image_name
    # Save the image with specific quality
    img_bgr_for_saving = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    cv2.imwrite(image_path, img_bgr_for_saving)

def harris_corner_detection(img, sigma, k):

```

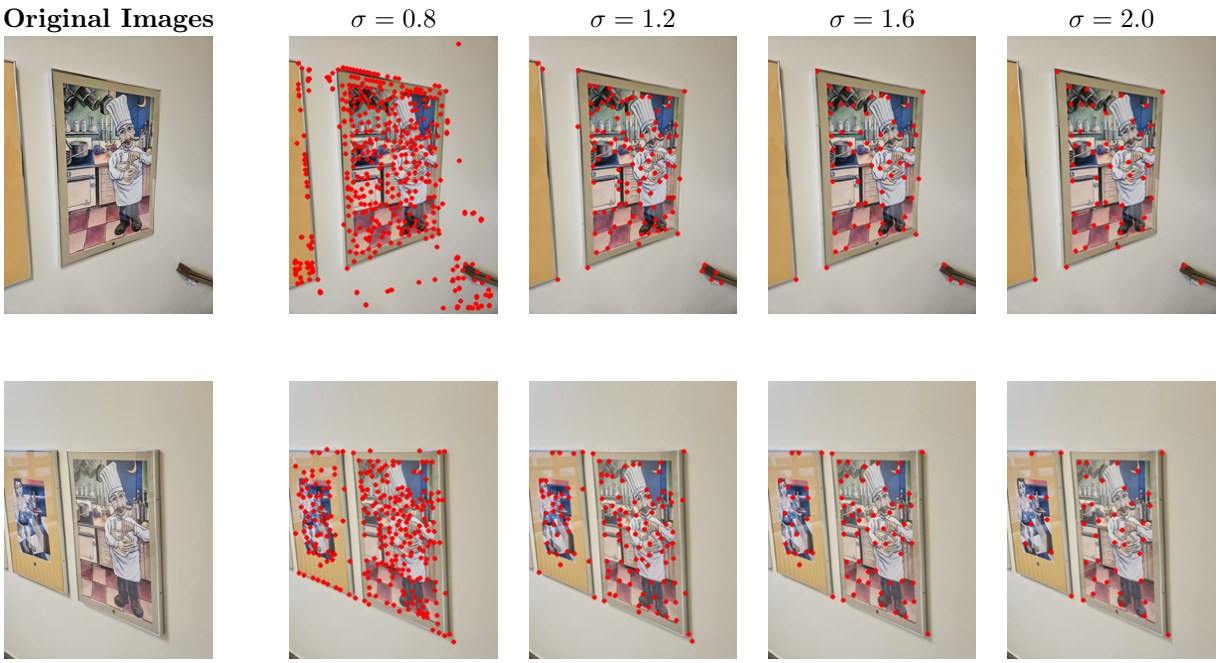


Figure 10: Original images (leftmost) and transformed images for different values of σ from the images of 'painting'.

```
    img_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    img_gray = img_gray.astype(np.float32)


    M = math.ceil(4*sigma)
    if M%2 != 0:
        M += 1

    # getting the haar kernels
    haar_x = np.ones((M,M))
    haar_x[:,:M//2] = -1

    haar_y = np.ones((M,M))
    haar_y[:M//2] = -1

    dx = cv2.filter2D(img_gray, -1, haar_x)
    dy = cv2.filter2D(img_gray, -1, haar_y)

    dxx = dx*dx
    dxy = dx*dy
    dyy = dy*dy
    ksize = int(5*sigma)
    sum_filter = np.ones((ksize,ksize))
    Sxx = cv2.filter2D(dxx, -1, sum_filter)
    Sxy = cv2.filter2D(dxy, -1, sum_filter)
    Syy = cv2.filter2D(dyy, -1, sum_filter)


    # Harris response ..
    det_M = Sxx*Syy - Sxy**2
```

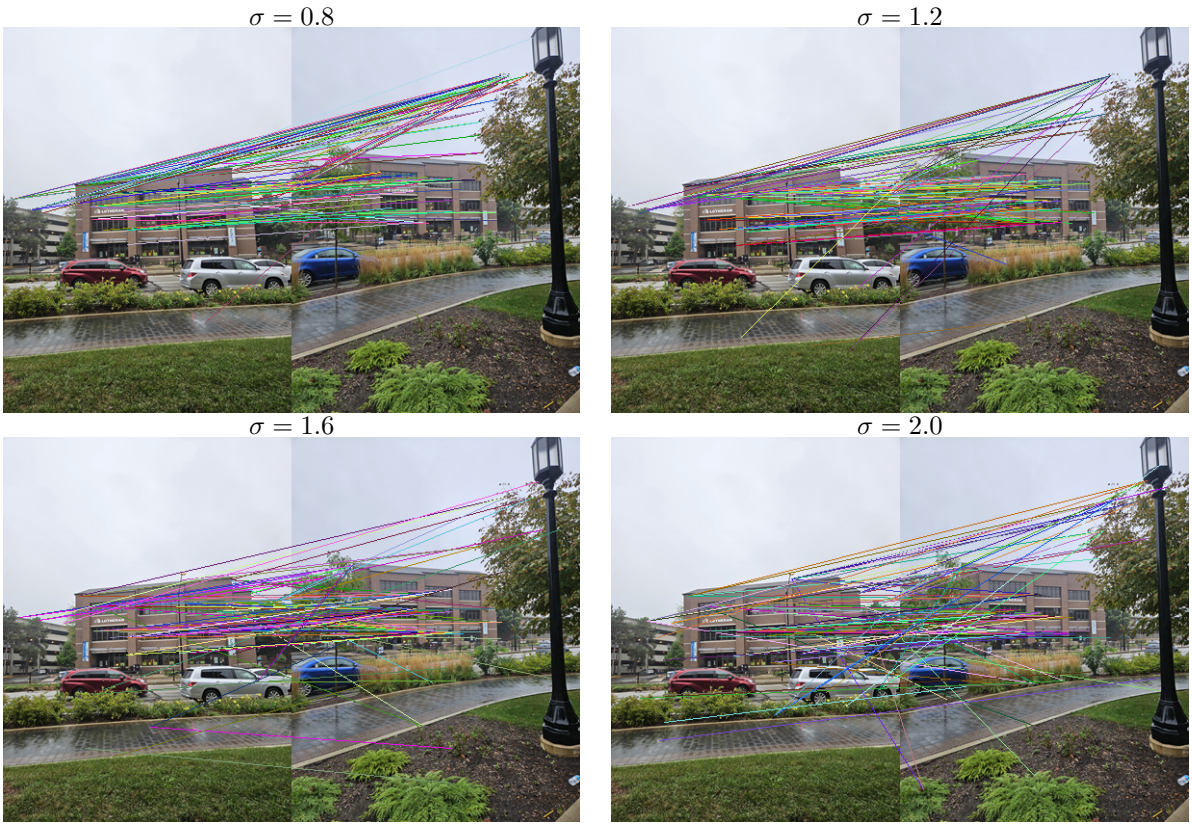


Figure 11: SSD correspondences for different values of σ from the set 'fuel'.

```
    trace_M = Sxx + Syy
    R = det_M - k*(trace_M**2)

    corners, R_norm = non_maximum_suppression(R, 2*ksize)
    return corners, R_norm


# Non-maximum suppression...
def non_maximum_suppression(R, k):
    """For every window of size k x k, retain only the pixel
    Args:
        R: Harris reponse
        k: window size
    """
    top_k = 500
    R_max = np.max(R)
    R_min = np.min(R)
    R_norm = (R-R_min)/(R_max - R_min)
    R_threshold = np.mean(R_norm)
    corners = []
    for i in range(k, R_norm.shape[0] - k):
        for j in range(k, R_norm.shape[1] - k):
            window = R_norm[i-k//2: i+k//2, j-k//2:j+k//2]
            local_max = np.max(window)
            if R_norm[i,j] == local_max and R_norm[i,j] > R_threshold:
                corners.append([i, j, R_norm[i,j]])
    corners = np.array(corners)
```

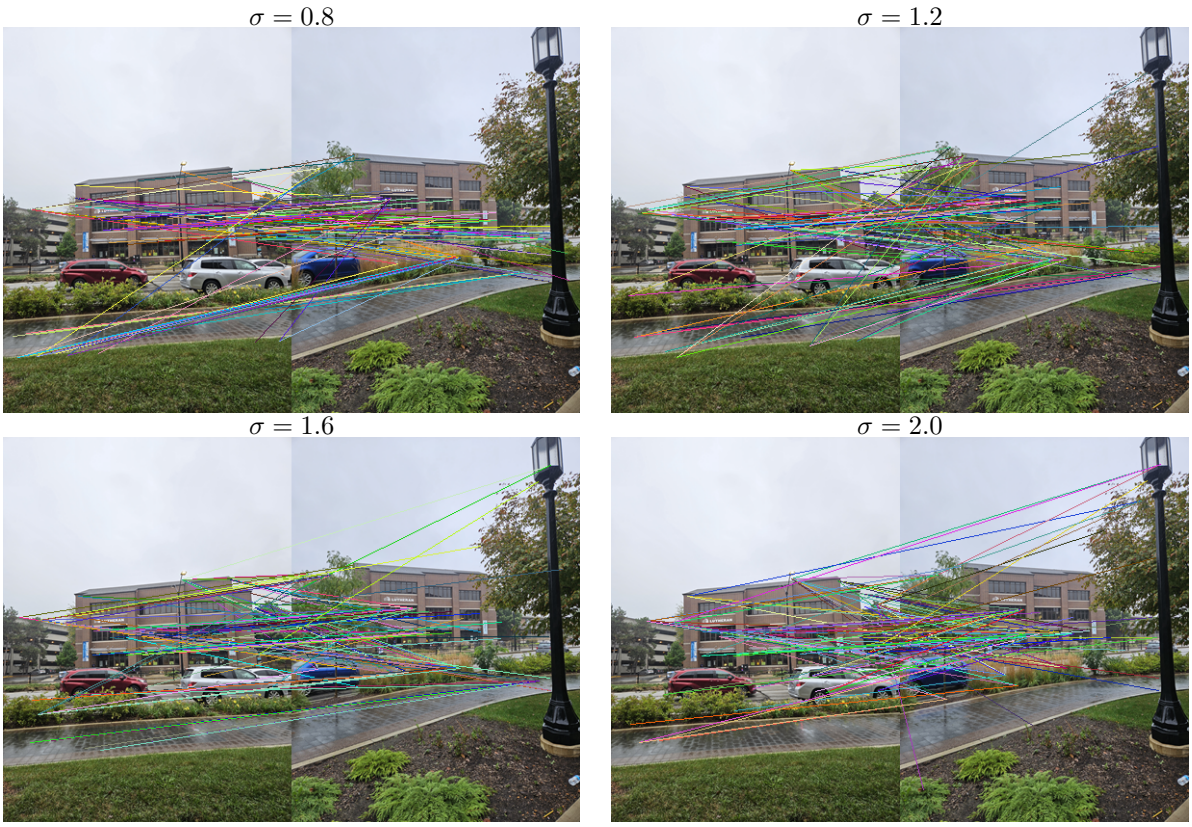


Figure 12: NCC correspondences for different values of σ from the set 'fuel'.

```
88      print(corners.shape)
89      if corners.shape[0] > top_k:
90          percentile = 100 - 100*top_k/corners.shape[0]
91          R_values = np.sort(corners[:,2])
92          threshold = np.percentile(corners[:,2], percentile)
93          mask = np.where(corners[:,2] > threshold)[0]
94          corners = corners[mask,:]
95          print(corners.shape)
96      return corners, R_norm
97
98  def add_corner_points_to_image(
99          img, corners,
100         color=(255,0,0),
101         radius=2,
102         thickness=2,
103         ):
104     # create an copy to avoid updating the original image.
105     output_image = np.copy(img)
106     for corner in corners:
107         # note the cv2.circle expects point as cartesian coordinates (x,y),
108         # whereas our coordinates of corners are saved as (row, col) index.
109         point = (int(corner[1]), int(corner[0]))
110         cv2.circle(output_image, point, radius=radius, color=color,
111              thickness=thickness)
112
113     return output_image
114
```

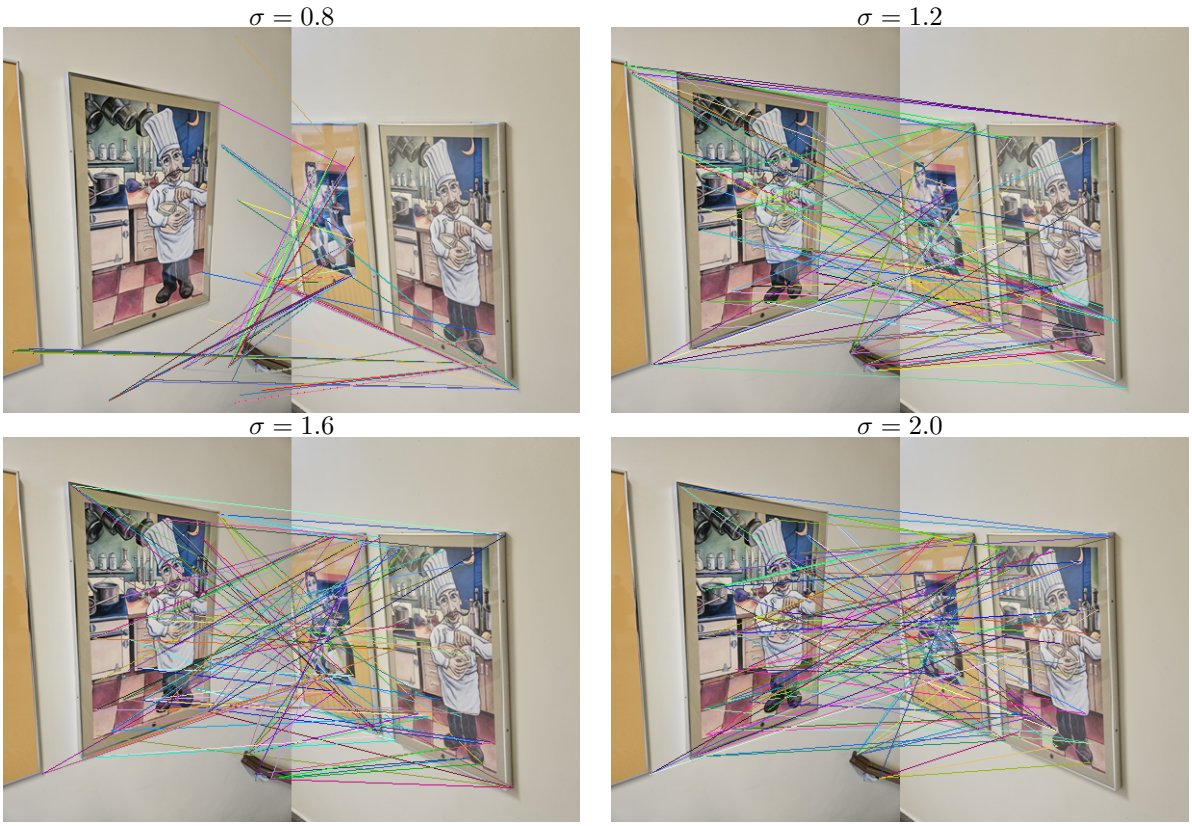


Figure 13: SSD correspondences for different values of σ from the set 'painting'.

```
115 def correspondence_using_SSD(
116         img1, img2, interest_points1, interest_points2, M=20, top_k=100):
117 
118     features1, interest_points1 = extract_features(img1, interest_points1, M
        )
119     features2, interest_points2 = extract_features(img2, interest_points2, M
        )
120     ssd = np.sum((features1[:,None,:] - features2[None,:,:])**2, axis=2)
121 
122     ssd_norm = (ssd - np.min(ssd))/(np.max(ssd) - np.min(ssd))
123     percentile = 100*top_k/ssd_norm.size
124     threshold = np.percentile(ssd_norm, percentile)
125     indices = np.where(ssd_norm < threshold)
126 
127     corresponding_points = {
128         'img1': interest_points1[indices[0]],
129         'img2': interest_points2[indices[1]],
130     }
131 
132     return corresponding_points
133 
134 def correspondence_using_NCC(
135         img1, img2, interest_points1, interest_points2, M=20, top_k=100
136         ):
137     features1, interest_points1 = extract_features(img1, interest_points1, M
        )
```

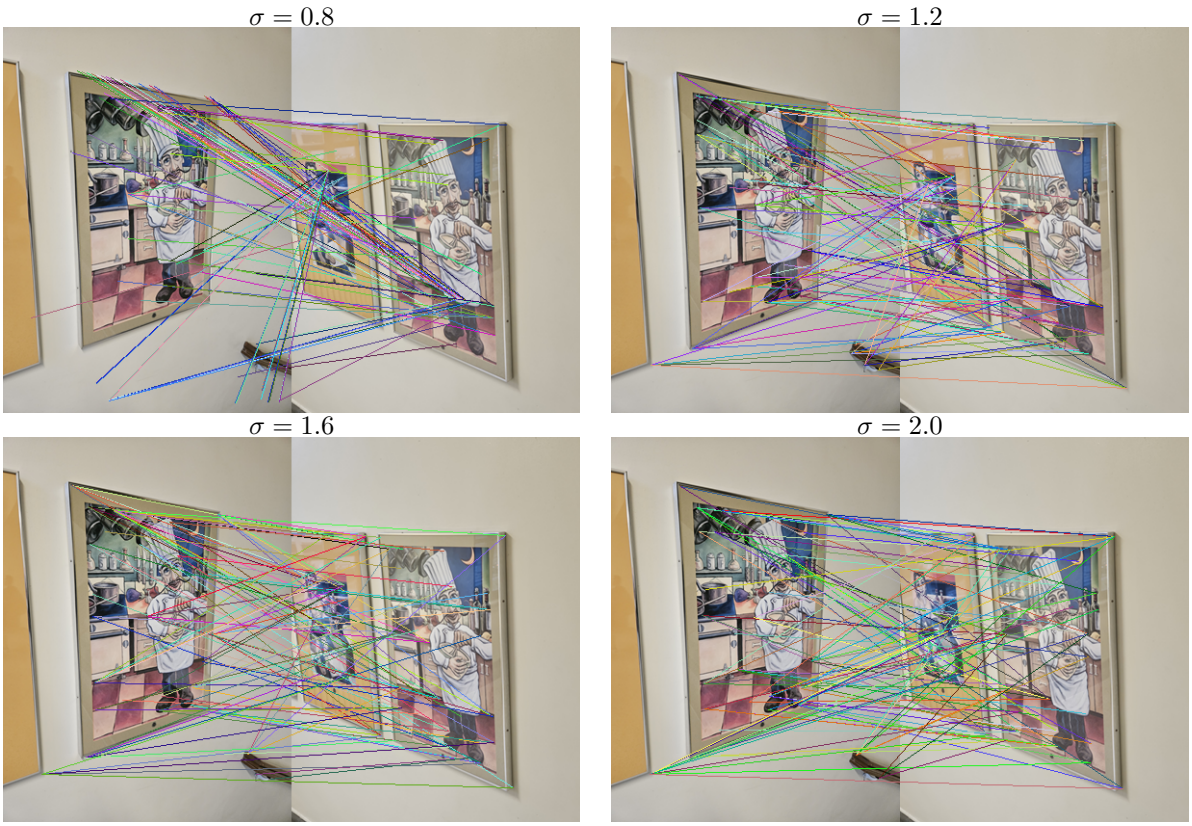


Figure 14: NCC correspondences for different values of σ from the set 'painting'.

```
138     features2, interest_points2 = extract_features(img2, interest_points2, M
            )
139
140     demean_f1 = features1 - np.mean(features1, axis=1)[:,None]
141     demean_f2 = features2 - np.mean(features2, axis=1)[:,None]
142
143     numerator = np.sum(demean_f1[:,None,:]*demean_f2[None,:,:], axis=2)
144     denom = np.sqrt(np.sum(demean_f1**2, axis=-1)[:,None]*np.sum(demean_f2
            **2, axis=-1)[None, :])
145
146     epsilon = 1e-6
147     ncc = numerator/(denom + epsilon)
148     ncc_dist = 1 - ncc
149     percentile = 100*top_k/ncc_dist.size
150     threshold = np.percentile(ncc_dist, percentile)
151     indices = np.where(ncc_dist < threshold)
152     corresponding_points = {
153         'img1': interest_points1[indices[0]],
154         'img2': interest_points2[indices[1]],
155     }
156
157     return corresponding_points
158
159 def extract_features(img, interest_points, M):
160     features = []
161     img_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
162     valid_interest_points = []
```

```
    for point in interest_points:
        i = int(point[0])
        j = int(point[1])
        if i >= M//2 and i< img_gray.shape[0]-M//2 and \
            j >= M//2 and j< img_gray.shape[1]-M//2:
            # print(img_gray[i-M//2:i+1+M//2, j-M//2:j+1+M//2].shape)
            features.append(img_gray[i-M//2:i+1+M//2, j-M//2:j+1+M//2].
                flatten())
            valid_interest_points.append(point)

    return np.array(features), np.array(valid_interest_points)

def display_corresponding_points(
        img1, img2, corresponding_points,
        thickness = 2,
        ):
    img_combined = np.concatenate([img1, img2], axis=1)
    half = corresponding_points['img1'].shape[0]
    for ind, (pt1, pt2) in enumerate(
            zip(corresponding_points['img1'], corresponding_points['img2'])):
        point1 = (int(pt1[1]), int(pt1[0]))
        point2 = (int(img1.shape[1]+pt2[1]), int(pt2[0]))
        # Randomly generate RGB values (each ranging from 0 to 255)
        color = (np.random.randint(0, 255), np.random.randint(0, 255), np.
            random.randint(0, 255))
        cv2.line(
            img_combined, point1, point2, color=color, thickness=thickness
            )
        if ind > half:
            break

    plt.imshow(img_combined)
    return img_combined


def get_kps_and_correspondences_using_harris_corner_detection(filename1,
    filename2, sigma=1, subdir='hw4'):

    k = 0.05
    M = 20
    top_k=100
    lw = 1

    # reading images...
    img1 = read_image(subdir, filename1)
    img2 = read_image(subdir, filename2)

    corners1, R_norm = harris_corner_detection(img1, sigma=sigma, k=k)
    corners2, R_norm = harris_corner_detection(img2, sigma=sigma, k=k)

    # adding interest points to the img
    img_w_kps1 = add_corner_points_to_image(img1, corners=corners1)
    img_w_kps2 = add_corner_points_to_image(img2, corners=corners2)

    save_image(img_w_kps1, subdir, f'harris_kps_sigma_{10*sigma:02.0f}_{
        filename1[:-4]}.png')
    save_image(img_w_kps2, subdir, f'harris_kps_sigma_{10*sigma:02.0f}_{
        filename2[:-4]}.png')
```

```
216
217     # correspondence using NCC
218     corresponding_points = correspondence_using_NCC(
219         img1, img2, corners1, corners2, M=M, top_k=top_k
220         )
221     combined_img = display_corresponding_points(img1, img2,
            corresponding_points, thickness=lw)
222     save_image(combined_img, subdir, f'
            harris_correspondence_using_NCC_sigma_{10*sigma:02.0f}_{filename1
            [:-5]}.png')
223
224     # correspondence using SSD
225     corresponding_points = correspondence_using_SSD(
226         img1, img2, corners1, corners2, M=M, top_k=top_k
227         )
228     combined_img = display_corresponding_points(img1, img2,
            corresponding_points, thickness=lw)
229     save_image(combined_img, subdir, f'
            harris_correspondence_using_SSD_sigma_{10*sigma:02.0f}_{filename1
            [:-5]}.png')
230
231
232
233 def get_kps_and_correspondences_using_SIFT(
234         filename1, filename2, subdir='hw4', top_k=100
235         ):
236     # reading images...
237     img1 = read_image(subdir, filename1)
238     img2 = read_image(subdir, filename2)
239
240     img1_gray = cv2.cvtColor(img1, cv2.COLOR_RGB2GRAY)
241     img2_gray = cv2.cvtColor(img2, cv2.COLOR_RGB2GRAY)
242
243     sift = cv2.SIFT_create()
244
245     kp1, des1 = sift.detectAndCompute(img1_gray, None)
246     kp2, des2 = sift.detectAndCompute(img2_gray, None)
247
248     bf = cv2.BFMatcher()
249     matches = bf.match(des1, des2)
250     matches = sorted(matches, key=lambda x: x.distance)
251
252     out_img = np.zeros_like(img2)
253     out_img = cv2.drawMatches(
254         img1, kp1, img2, kp2, matches[:top_k], out_img,
255         flags=2
256     )
257     save_image(out_img, subdir, f'SIFT_{filename1[:-5]}.png')
258
259
260
261 if __name__ == "__main__":
262
263     image_pairs = {
264         'pair_1': ('hovde_2.jpg', 'hovde_3.jpg'),
265         'pair_2': ('temple_1.jpg', 'temple_2.jpg'),
266         'pair_3': ('painting_1.png', 'painting_2.png'),
267         'pair_4': ('fuel_1.png', 'fuel_2.png'),
268     }
```

```
269
270     sigmas = [0.8, 1.2, 1.6, 2.0]
271
272     for pair, (filename1, filename2) in image_pairs.items():
273         print(f"Running for {pair}")
274         for sigma in sigmas:
275             get_kps_and_correspondences_using_harris_corner_detection(
276                 filename1, filename2, sigma=sigma
277             )
278
279         get_kps_and_correspondences_using_SIFT(filename1, filename2, 'hw4')
```

Listing 1: Example Python code

Hovde

Temple

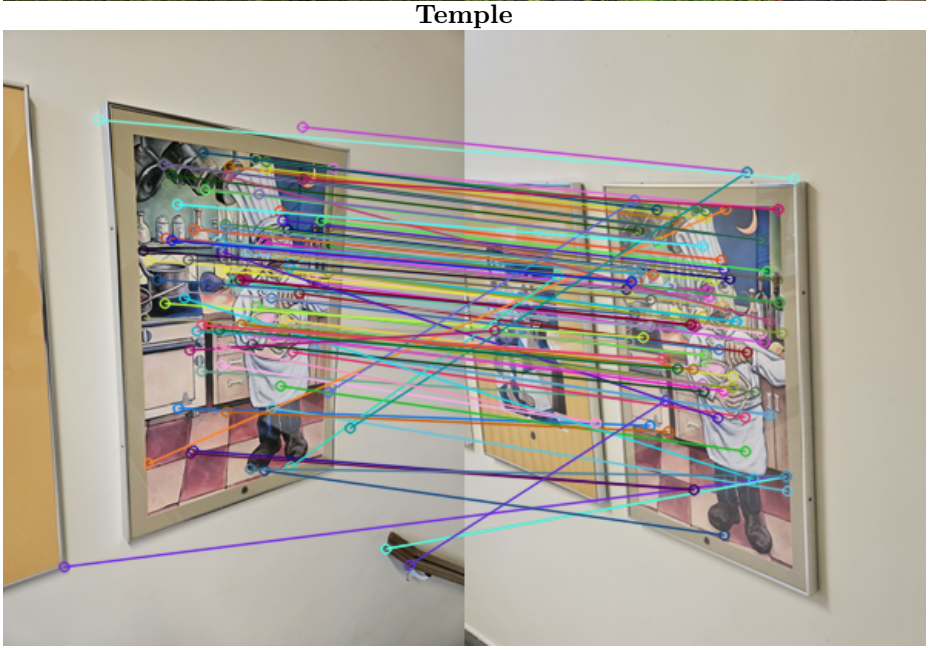
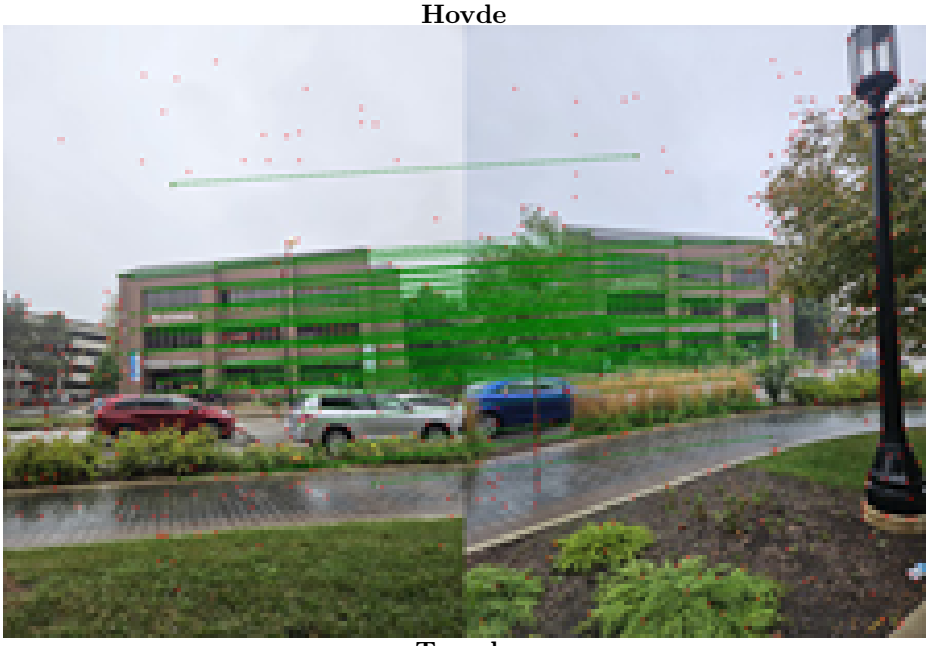


Figure 15: Interest points and correspondences using SIFT for the 'Fuel' and 'painting' pairs.

Hovde



Temple

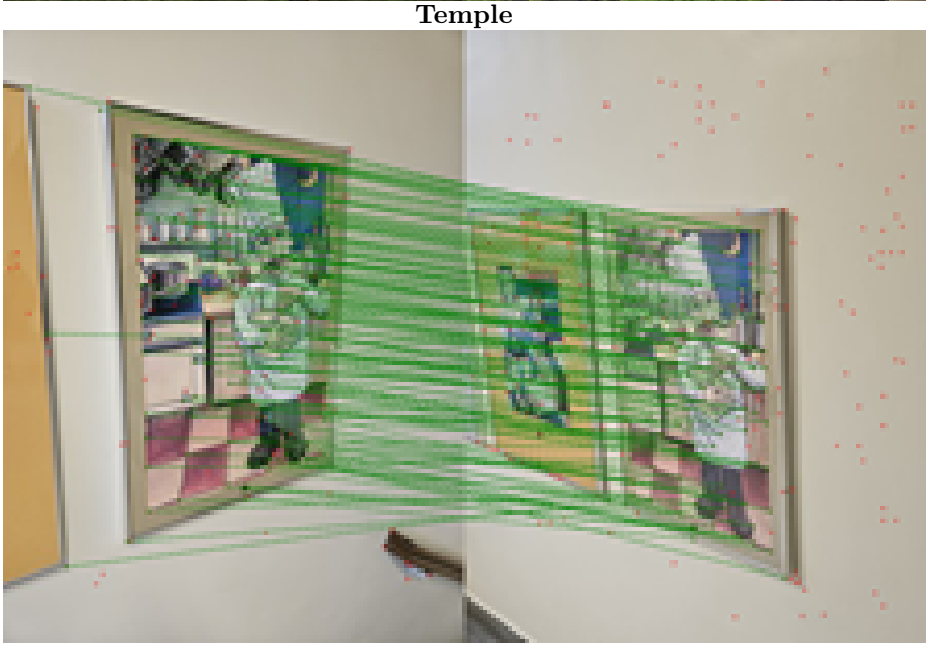


Figure 16: Result of using SuperPoint and SuperGLue for the 'Hovde' and 'Temple' pairs.